



Compiler Design

Published By:



Physics Wallah

ISBN: 978-93-94342-39-2

Mobile App: Physics Wallah (Available on Play Store)



Website: www.pw.live

Email: support@pw.live

Rights

All rights will be reserved by Publisher. No part of this book may be used or reproduced in any manner whatsoever without the written permission from author or publisher.

In the interest of student's community:

Circulation of soft copy of Book(s) in PDF or other equivalent format(s) through any social media channels, emails, etc. or any other channels through mobiles, laptops or desktop is a criminal offence. Anybody circulating, downloading, storing, soft copy of the book on his device(s) is in breach of Copyright Act. Further Photocopying of this book or any of its material is also illegal. Do not download or forward in case you come across any such soft copy material.

Disclaimer

A team of PW experts and faculties with an understanding of the subject has worked hard for the books.

While the author and publisher have used their best efforts in preparing these books. The content has been checked for accuracy. As the book is intended for educational purposes, the author shall not be responsible for any errors contained in the book.

The publication is designed to provide accurate and authoritative information with regard to the subject matter covered.

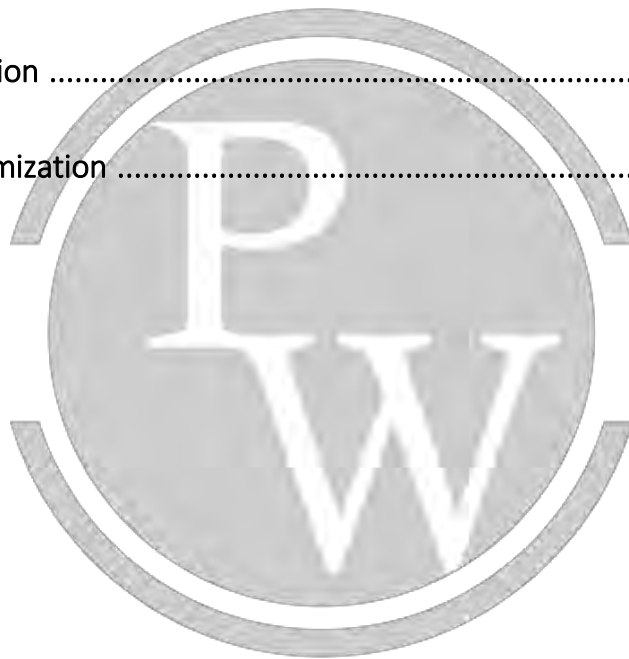
This book and the individual contribution contained in it are protected under copyright by the publisher.

(This Module shall only be Used for Educational Purpose.)

Compiler Design

INDEX

1.	Introduction and Lexical Analysis	8.1 – 8.2
2.	Parsing	8.3 – 8.11
3.	Lexical Analysis	8.12 – 8.13
4.	Syntax Directed Transaction	8.14 – 8.16
5.	Intermediate, Code Optimization	8.17 – 8.23



1

INTRODUCTION AND LEXICAL ANALYSIS

1.1 CD Introduction

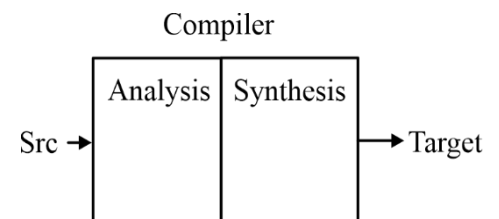
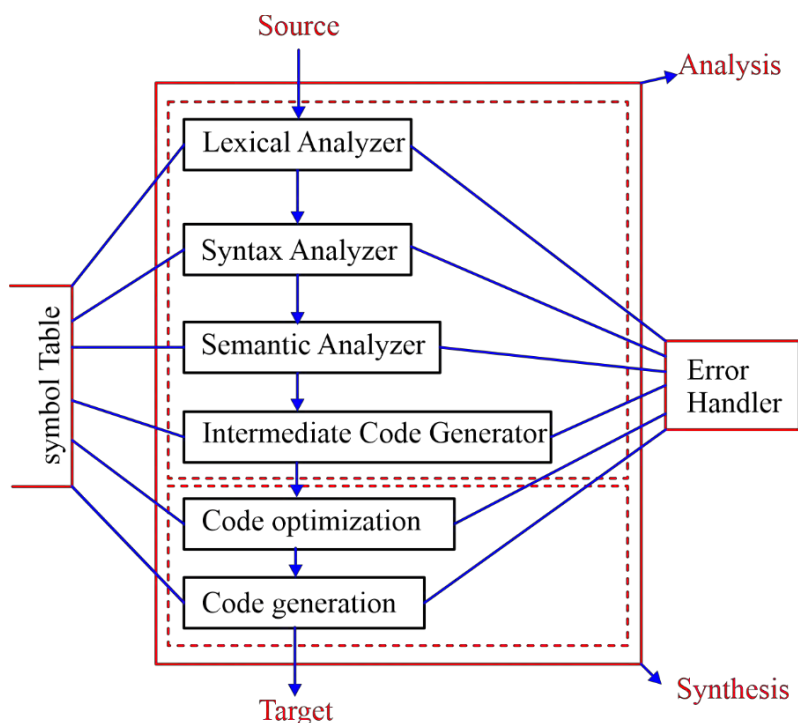
1.1.1 Definition

Convert High Level Language to Low Level Language.



- High Level Language can perform more than one operation in a single Statement.
- Low Level Language can perform at most one operation in a statement.

1.2 Analysis and Synthesis model of Compiler



- There are 6 phases of the Compiler



1. Lexical Analyzer:

- Program of DFA, it checks for spelling mistakes of program.
- Divides source code into stream of tokens.

2. Syntax Analyzer:

- Checks grammatical errors of the program (Parser).
- Parser is a DPDA.

3. Semantic Analyzer:

Checks for meaning of the program.

Example:

Type miss match, stack overflow

4. Intermediate Code Generation:

- This phase makes the work of next 2 phases much easier.
- Enforces reusability and portability.

5. Code optimization:

- Loop invariant construct
- Common sub expression elimination
- Strength Reduction
- Function in lining

Deadlock elimination

6. Symbol Table:

- (1) Data about Data (Meta data)
- (2) Data structure used by compiler and shared by all the phrase.



2

PARSING

2.1 CD - Grammar

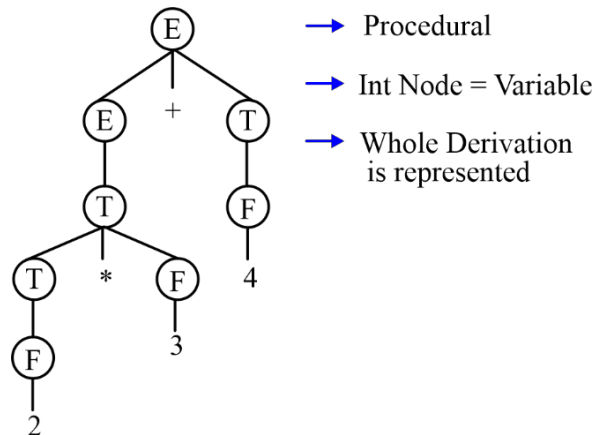
- In compiler we only use: Type – 2 (CFG) and Type – 3 Regular grammar.
- Compiler = Program of Grammar
- Compiler = Membership algorithm
- Every programming Language is Context Sensitive Grammar (Context Sensitive Language)

2.1.1 Parse Tree and Syntax Tree

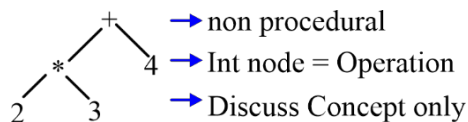
G: $E \rightarrow E + T / T$ $E \rightarrow E + T \rightarrow T + T \rightarrow T * F + T \rightarrow F * F + T \rightarrow 2 * F + T \rightarrow 2 * 3 + T \rightarrow 2 * 3 + F$
 $T \rightarrow T * F / F$

$E \rightarrow 2 * 3 + 4$

Parse Tree:

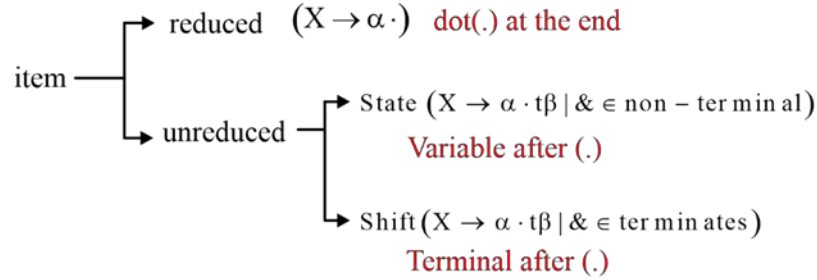


Syntax Tree:



- To check to priority / Associativity:
Randomly derive till you have enough operators, then check which one is done first.
- If priority of 2 operators is same and both Left and Right associative → **Ambiguous Grammar** [Useless]

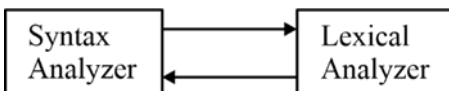
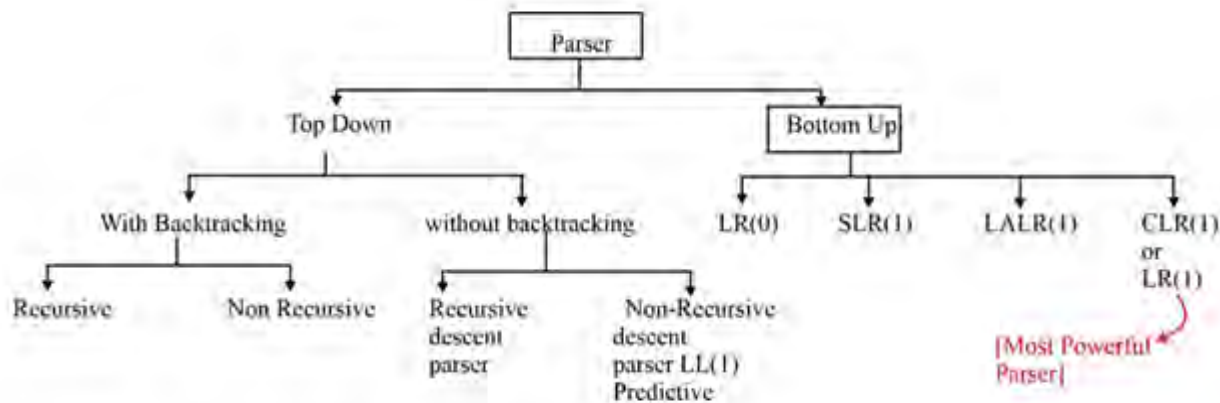
2.1.2 Types of Parsers in Bottom Up: [CD - Parser]



2.2 CD-Syntax Analysis / Parsing

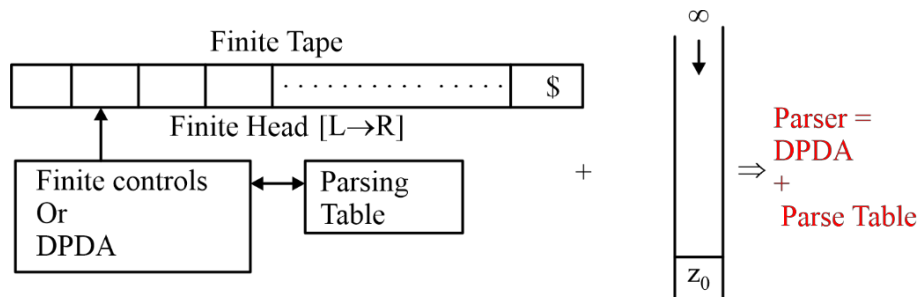
Grammatical Errors are checked by the help of parsers.

- Parsers are basically DPDA



- All of these parsers are table driven.

2.2.1 Mathematical Model of Parser



- Parsers generate Parse Tree, for a given string by the given grammar.

2.2.2 Top-Down Parser (LL (1))

- It uses LMD and is equivalent to DFS in graph.

2.2.3 Algorithm to construct Parsing Table

1. Remove Left Recursion if any.
 2. Left Factor [Remove Common Prefix]
 3. Find first and follow set
 4. Construct the table.
- If we increase the look ahead symbol:

2.2.4 Removal of common Prefix: (Left Factor)

1. $S \rightarrow a \mid a b \mid a A$ $S \rightarrow a Y$
 $Y \rightarrow \epsilon \mid b \mid A$
2. $A \rightarrow a b A \mid a A \mid b$
 $A \rightarrow a \times \mid b$ $A \rightarrow a \times \mid b$
 $X \rightarrow b A \mid A$ $X \rightarrow b A \mid a x \mid b$
 $A \rightarrow a X \mid b$
 $X \rightarrow a X \mid b$
 $Y Y \rightarrow A \mid \epsilon$

2.2.5 First and Follow

- First Set $\rightarrow$ extreme Left terminal from which the string of that variable starts.
 $\rightarrow$ it never contains variable, but may contain ' ϵ '.
 $\rightarrow$ we can always find the first of any variable.
- Follow set $\rightarrow$ Follow set contains terminals and \$.
It can never contain variable and ' ϵ '.
How to find follow set?
 1. Include \$ in follow of start variable.
 2. If production is of type $\rightarrow$
 $A \rightarrow \alpha B \beta$ $\alpha \beta \rightarrow$ strings of grammar symbol].
 $\text{follow}(B) = \text{first}(\beta)$.
If, $\beta \rightarrow \epsilon$, i.e. $A \rightarrow \alpha B$, then $\text{follow}(B) = \text{follow}(A)$.
- Productions like: $A \rightarrow \alpha A$ gives no follow set.

2.2.6 Example of first and follow set

- $S \rightarrow AB \mid CD$
 $A \rightarrow aA \mid a$
 $B \rightarrow bB \mid b$
 $C \rightarrow cC \mid c$
 $D \rightarrow dD \mid d$

	First	Follow
S	a, c	\$
A	a	b
B	b	\$
C	c	D
D	D	\$

2.2.7 Entity into Table: Top Down

- No of Rows = number of unique variable in Grammar.
 - No of columns = [Terminals + \$]
 - For a Variable (Row) fill the column (Terminal) if its P, there in its first with the production required.
 - If ϵ is in first put $V \rightarrow \epsilon$ under \$ and its follow.
- If any cell has multiple items, then its not possible to have LL (1) Parser, since that will be ambiguous.
 - In top down we do: Derivation
In Bottom up we do: Production.

Question: Construct LL (1) Parsing Table for the given Grammar: $E \rightarrow E + T \mid T$; $T \rightarrow T * F \mid F$; $F \rightarrow (E) \mid id$; $\Rightarrow G_0$

$\Rightarrow$ Removing Left Recursion:

$E \rightarrow TE'$

$E' \rightarrow + TE' \mid \epsilon$

$T \rightarrow FT' \quad G_1$

$T' \rightarrow * FT' \mid \epsilon$

$F \rightarrow (E) \mid id$

	First	Follow
E	C, id	\$,)
E'	+, ϵ	\$,)
T	C, id	+, \$,)
T'	*, ϵ	+, \$,)
F	C, id	*, +, \$,)

- Left Factoring not Required:
Construction of Table: [LL (1)]

	+	*	(	)	id	\$
E	Error	Error	$E \rightarrow TE'$	Error	$E \rightarrow TE'$	Error
E'	$E' \rightarrow TE'$	Error	Error	$E' \rightarrow \epsilon$	Error	$E' \rightarrow \epsilon$
T	Error	Error	$T \rightarrow FT'$	Error	$T \rightarrow FT'$	error
T'	$T' \rightarrow \epsilon$	$T' \rightarrow * FT'$	Error	$T' \rightarrow \epsilon$	Error	$T' \rightarrow \epsilon$
F	Error	Error	$F \rightarrow (E)$	Error	$F \rightarrow id$	error

- Since for G_1 , Table constructed with no multiple entries. Hence successfully completed. Hence G_1 is LL (1)

Question: Construct LL (1) Parsing Table for the following Grammar:

$S \rightarrow L = R \mid R$; $L \rightarrow * R \mid id$; $R \rightarrow L \Rightarrow G_0$

Solution: Left Factoring:

$$\begin{aligned}
 S &\rightarrow L = R \mid L & S &\rightarrow LX \\
 L &\rightarrow *R \mid id & \Rightarrow & \times \Rightarrow = R \mid \in \\
 R &\rightarrow L & L &\rightarrow *R \mid id \\
 & & R &\rightarrow L
 \end{aligned}
 \Rightarrow G_1$$

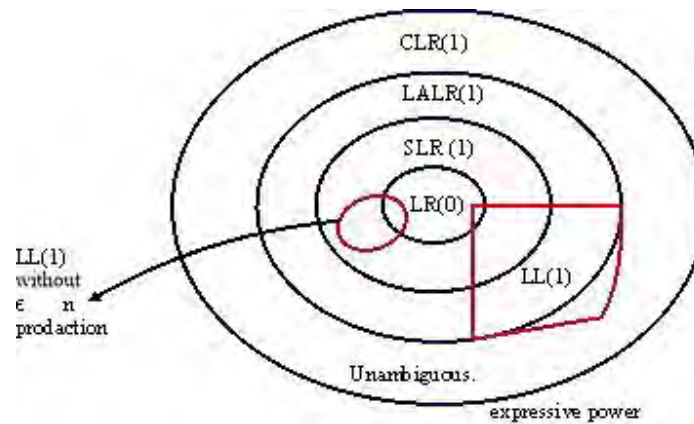
	First	Follow
S	*, id	\$
×	=, ∈	\$
L	*, id	\$
R	*, id	\$

Construction of Table:

	*	=	Id	\$
S	S→LX	Error	S→L X	Error
L	L→*R	Error	L→id	Error
R	R→L	Error	R→L	Error
×	Error	X→ = R	Error	X→∈

- G_1 is a LL (1) Grammar.

2.2.8 Hierarchy of Parsers: [for ϵ - free Grammar]



- For ϵ - producing grammars every LL (1) may not be LALR (1).

Note:

We can't construct any parser for ambiguous grammar. Except: Operator precedence, parser possible for some ambiguous grammar.

Example:

$G: S \rightarrow a S a \mid b S b \mid a \mid b$ (Unambiguous but no parser) $L(G) = \omega(a + b) \omega R$
(Odd palindrome)

- Every RG is not LL (1) as it may be ambiguous, or Recursive or Common Prefix.
- Parsers exists only for the grammar if its Language is DCFL.
- There are some grammars whose Language is DCFL but no parser is possible for it.

2.4.9 Operator Precedence Grammar

Format: (1) No 2 or more variable side by side
(2) No $\in$ production.

Example:

$E \rightarrow E + T \mid T$ $E \rightarrow E * E$
 $T \rightarrow T * F \mid F$ or $E \rightarrow E * E$ or $S \rightarrow aSa \mid bSb \mid a \mid b$
 $F \rightarrow (E) \mid id$ $E \rightarrow a / b$ O.G.
 O.G. O.G.
 Or
 $S \rightarrow AB$
 $A \rightarrow aA \mid \in$ } Not O.G.
 $B \rightarrow bB \mid \in$ }

2.2.10 Checking LL (1) without Table

$A \rightarrow \alpha_1 \mid \alpha_2 \mid \alpha_3$ then $\rightarrow$ $A \rightarrow \alpha_1 \mid \alpha_2 \mid \alpha_3 \mid \in$
 $\text{first}(\alpha_1) \cap \text{first}(\alpha_2) = \emptyset$ $\text{first}(\alpha_1) \cap \text{first}(\alpha_2) = \emptyset$
 $\text{first}(\alpha_1) \cap \text{first}(\alpha_3) = \emptyset$ $\text{first}(\alpha_1) \cap \text{first}(\alpha_3) = \emptyset$
 $\text{first}(\alpha_2) \cap \text{first}(\alpha_3) = \emptyset$ $\text{first}(\alpha_2) \cap \text{first}(\alpha_3) = \emptyset$
 $\text{follow}(A) \cap \text{first}(\alpha_1) = \emptyset$
 $\text{follow}(A) \cap \text{first}(\alpha_2) = \emptyset$
 $\text{follow}(A) \cap \text{first}(\alpha_3) = \emptyset$

2.2.11 Bottom-UP Parser

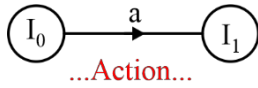
- It uses RMD in reverse and has no problem with:
 - (a) Left recursion (b) Common Prefix
- SLR (1) LR (0) CLR (1) LR (1)
LR (0) items LALR (1) items
LR (1) = LR(0) + 1 Lookahead
- No parser possible for ambiguous grammar.
- There are some unambiguous grammars for which, there are no Parser.

2.2.12 Basic Algorithm for Construction

- Augment the grammar and expand it, and give numbers to it
- Construct LR (0) or LR (1) item set.
- From that fill the entries in the Table Accordingly.

2.2.13 Types of Entries

(1) **Shift Entries:** Transitions or Terminals



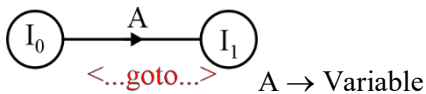
Entry:

	a
I ₀	S ₁

$a \in \text{Terminal } I_0, I_1: \text{Item sets.}$

- Shift entries are same for all Bottom – up Parser.

(2) **State Entry:** Transition or non – terminal (variable)



Entry:

	A
I ₀	1

- Same for all Bottom-up Parser.

(3) **Reduce Entity:** done for each separate production in the item set of type:

$$i > \times \rightarrow \alpha$$

where, $i \rightarrow \text{Production Number}$

$\times \rightarrow \text{Producing Variable}$

$\alpha \rightarrow \text{Grammar String}$

(a) LR (0) Parser:

Put Ri in every cell of the set-in action table (**ALL**).

(b) SLR (1) Parser:

Put Ri only in the follow(x) form the Grammar. (**Follow(x)**)

(c) LALR (1) and CLR (1):

Put Ri only in the look ahead of the production (**Lookaheads**)

2.2.14 Conflicts

LR (0) Parser:

SR: Shift Reduce Conflict

$S \rightarrow X \rightarrow \alpha \cdot t\beta \Rightarrow$ then SR

$R \rightarrow Y \rightarrow \delta \cdot$ conflict

RR: Reduce Reduce conflict

$R \rightarrow X \rightarrow \alpha \cdot \Rightarrow$ then RR

$R \rightarrow X \rightarrow \beta \cdot$

SLR (1) Parser:

SR:

$S \rightarrow X \rightarrow \alpha \cdot t\beta \quad \$ \Rightarrow$ then SR

$R \rightarrow Y \rightarrow \delta \cdot$ conflict

$t \in \text{follow}(Y)$

RR:

$X \rightarrow \alpha \cdot \quad \$ \Rightarrow$ then RR

$Y \rightarrow \beta \cdot$

$\text{Follow}(X) \cap \text{follow}(Y) \neq \emptyset$

LALR (1) and CLR (1): Same as SLR (1), but instead

SR:

$X \rightarrow \alpha \cdot t\beta \cdot L_1 \quad \$ \Rightarrow$ then SR

$Y \rightarrow \delta \cdot L_2$ conflict

$T \in L_2$

RR:

$X \rightarrow \alpha \cdot, L_1 \quad \$$ then RR

$Y \rightarrow \beta \cdot, L_2$ conflict

$L_1 \cap L_2 \neq \emptyset$

2.4.15 Inadequate Static

A static having ANY conflict is called a conflicting state or independent state.

Note:

The state $S' \rightarrow S$ or $S' \rightarrow S, \$$ is accepted state, and this is not a reduction.

- The only difference between CLR (1) and LALR (1) is that, the states with the similar items, but different Lookaheads are merged together to Reduce space.

Important Points

- If CLR (1) doesn't have any conflict, then conflict may or may not arise after merging in LALR (1).
- If LALR (1) has SR – conflict, then we can conclude that CLR (1) also has SR – Conflict.
- LALR (1) has SR – conflict if and only if CLR (1) also has SR.

- We can construct parser for every unambiguous regular grammar: [CLR (1) Parser].

	L Left to right scan of i/p	R Using reverse RMD	(0) No Lookahead	
S	L	R	(1)	
Simple	L to R Scan	Using Reverse RMD	Lookahead	
L	A	L	R	(1)
Look	Ahead	L to R Scan	Revers RMD	Lookahead
C	L	R	(1)	
Canonical	L to R Scan	reverse RMD	Lookahead	

Very Important Point:

LALR (1) Parser can Parse non LALR (1) grammar, when only non-SR – Conflict by favouring shift over reduce.

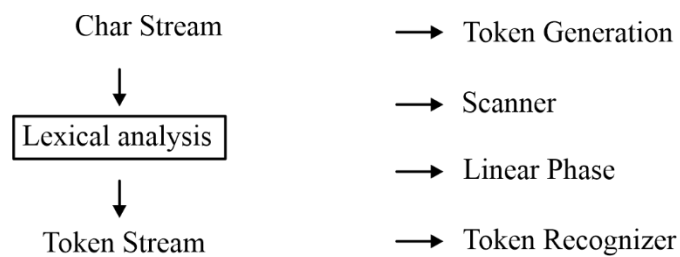
Example: $E \rightarrow E + E \mid E * E \mid id \mid 2 + 3 * 5 \Rightarrow E + E \cdot * 5$



3

LEXICAL ANALYSIS

3.1 Lexical Analysis



3.1.1 Definition

Scan the whole program, char by char and produces the corresponding token.

- Also produces /Lexical Errors (if any).
- Functions of Lexical Analyzer.
 - (1) Scans all the character of the program.
 - (2) Token recognizer.
 - (3) Ignores the comment & spaces.
 - (4) Maximal Munch Rule [Longest Prefix Match].

Note:

The Lexical Analyser uses, the Regular Expression.
Prioritization of Rules.
Longest Prefix match .

- Lexeme → smallest unit of program or Logic.
- Token → internal representation of Lexeme.

3.1.2 Types of Token

- (1) Identifier
- (2) Keywords
- (3) Operators
- (4) Literals/Constants
- (5) Special Symbol

3.1.3 Token Separation

- (1) Spaces
- (2) Punctuation

3.1.4 Implementation

- LEX tool $\Rightarrow$ Lex.yy.c
- All identifiers will have entry in symbol Table/ LA, gives entries into the symbol Table.
Regular Expression $\rightarrow$ DFA $\rightarrow$ Lexical Analyzer

3.1.5 Find number of Tokens

- (1) `void main () {`
 `Printf("gate");`
 `}`
 {11 Tokens}
- (2) `int x, *P;`
 `x = 10; p = &x; x++;`
 [18 Tokens]
- (3) `int x;`
 `x = y;`
 `x == y;`
 [11 Tokens]
- (4) `int 1x23;` [Lexical Error]
- (5) `char ch = 'A';`
 [5 Token]
- (6) `char ch = 'A;`
 Lexical Error.
- (7) `char *p = "gate";`
 [6 Tokens]
- (8) `char * p = "gate;`
 Error.
- (9) `int x = 10;`
 `/* comment x = x + 1; ERROR`
 `x = x + 1; [11 Tokens]`



4

SYNTAX DIRECTED TRANSACTION

4.1 Syntax Directed Transaction

CFG + Transition } SDT
Syntax + Transition

SDT: CFG + Transition → 1) Meaning 2) Semantic

4.1.1 Application of SDTL

- (1) Used to perform Semantic Analysis.
- (2) Produce Parse Tree.
- (3) Produce intermediate representation.
- (4) Evaluate an expression.
- (5) Convert infix to prefix or postfix.

Example: $S \rightarrow S_1 S_2$ [$S.count = S_1.count + S_2.count$]

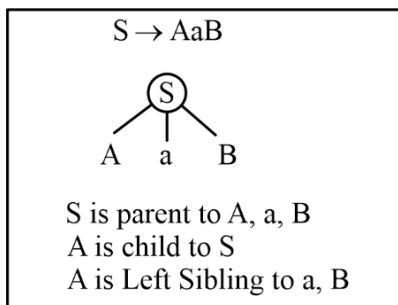
$S \rightarrow (S_1)$ [$S.count = S_1.count + 1$]

$S \rightarrow \epsilon$ [$S.count = 0$]

- Count is an attribute for non-terminal

4.1.2 Attributes

- (1) Inherited Attribute
- (2) Synthesized Attribute



- (1) Inherited Attribute: (RHS)

$S \rightarrow \boxed{A} B$ { $A.x = f(B.x \mid S.x)$ }

- The computation at any node (non-terminal) depends on parent or siblings (S).

- In above Example, x is inherited attribute.

(2) Synthesized Attribute: (LHS)

$$S \rightarrow A B \{S.x = f(A.x \mid B.x)\}$$

x is synthesized attribute.

The computation at any node (non-terminal) depends on children.

4.1.3 Identifying Attribute Type

- Always check every Translation.

- | | | |
|---|---|--|
| 1) $D \rightarrow T : L ; \{L.Type = T.Type\}$ inherited
$L \rightarrow L1, id ; \{L1.Type = L.Type\}$ inherited
$L \rightarrow id$
$T \rightarrow integer \{T.Type = int\}$ synthesized | } | Type is neither inherited nor synthesized |
| 2) $E \rightarrow E1 + T \{E.val = E1.val + T.val\}$ synthesized
$E \rightarrow T \{E.val = T.val\}$ Synthesized
$T \rightarrow T1 - F \{T.val = T1.val - F.val\}$ Synthesized
$F \rightarrow id \{F.val = id.val\}$ synthesized | } | Val is synthesized attribute |
| 3) $S \rightarrow AB \{Aa = B.x; S.y = A.x\}$ x is inherited y is synthesized
$A \rightarrow a \{A.y = a\}$ y is synthesized
$B \rightarrow b \{B.y = a.y\}$ y is synthesized | } | $x \Rightarrow$ inherited
$y \Rightarrow$ synthesized |
| 4) $D \rightarrow T L \{L.in = T.type\}$ inherited(in)
$T \rightarrow int \{T.type = int\}$ /synthesized
$L \rightarrow id \{Add\ type(id.entry, L.in)\}$ | } | $in \Rightarrow$ inherited
$type \Rightarrow$ synthesized |

4.1.4 Syntax Directed Definitions (SDDs): (Attribute Grammar)

(1) L-Attributed Grammar

- Attribute is synthesized or restricted inherited. (1) Parent 2) Left sibling only).
- Translation can be appended anywhere is RHS of production.
- Example: $S \rightarrow AB \{A.x = S.x + 2\}$
 or, $S \rightarrow AB \{B.x = f(A.x \mid S.x)\}$
 or, $S \rightarrow AB \{S.x = f(A.x \mid B.x)\}$
- Evaluation: In Order (Topological).

(2) S-Attributed Grammar:

- Attribute is synthesized only.
- The transaction is placed only at the end of production.
- Example: $S \rightarrow AB \{S.x = f(Ax \mid Bx)\}$.
- **Evaluation:** Reverse RMD (Bottom-Up Parsing).

4.1.5 Identify SDD

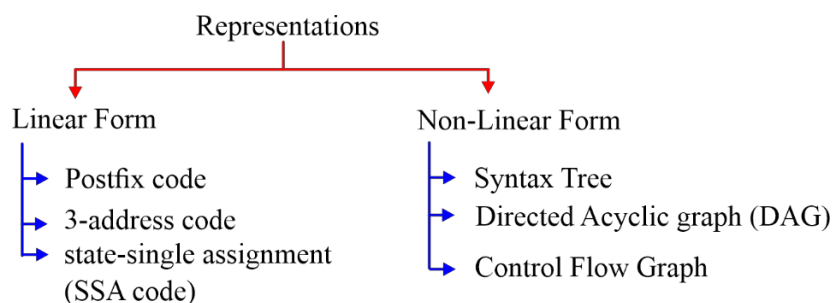
- (1) $E \rightarrow E_1 + E_2$ {E.type = if (E₁.type == int & & E.type == int) then int} Synthesizer else type – error.
 $E \rightarrow id$ {E.type = Lookup (id.entry)} synthesizer.
- type is synthesized, hence S-attribute and also L-attributed Grammar.
 - Every S-attributed Grammar is also L-attributed Grammar.
 - For L-attributed Evaluation, use the In-order of annotated Parser Tree.
 - For S-attributed, reverse of RMD is used.
- find RMD Order.
 → Consider its reverse.



5

INTERMEDIATE, CODE OPTIMIZATION

5.1 Introduction



Example Expression:

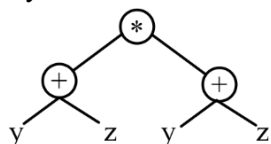
$(y + z) * (y + z)$

Post fix $\rightarrow$ $yz + yz + *$

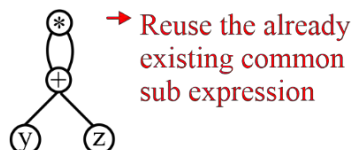
SSA $\rightarrow$ $t_1 = y + z$
 $t_2 = t_1 * t_2$ $\Rightarrow f_1$ and f_2 cannot be reassigned

3AC $\rightarrow$ $t_1 = y + z$
 $t_2 = t_1 \times t_2$

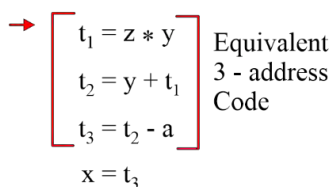
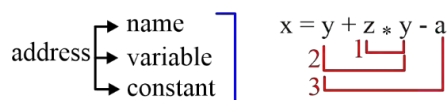
Syntax Tree $\rightarrow$



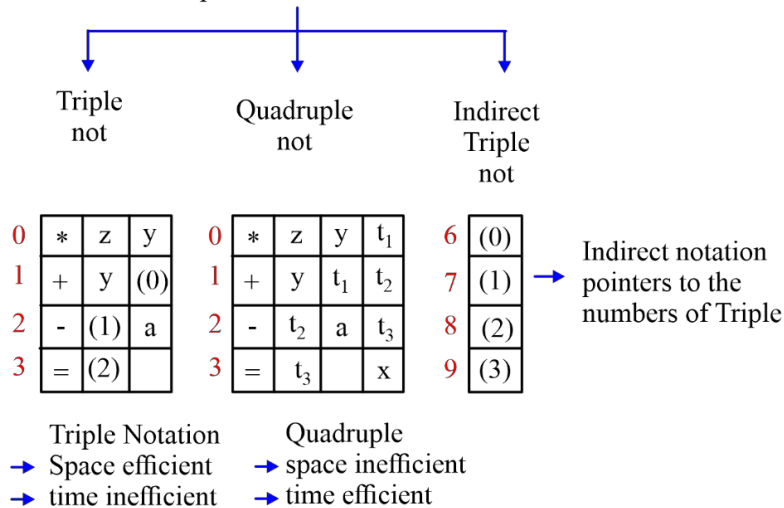
DAG $\rightarrow$



3-address Code: Code in which, at most 3 addresses. [including LHS]

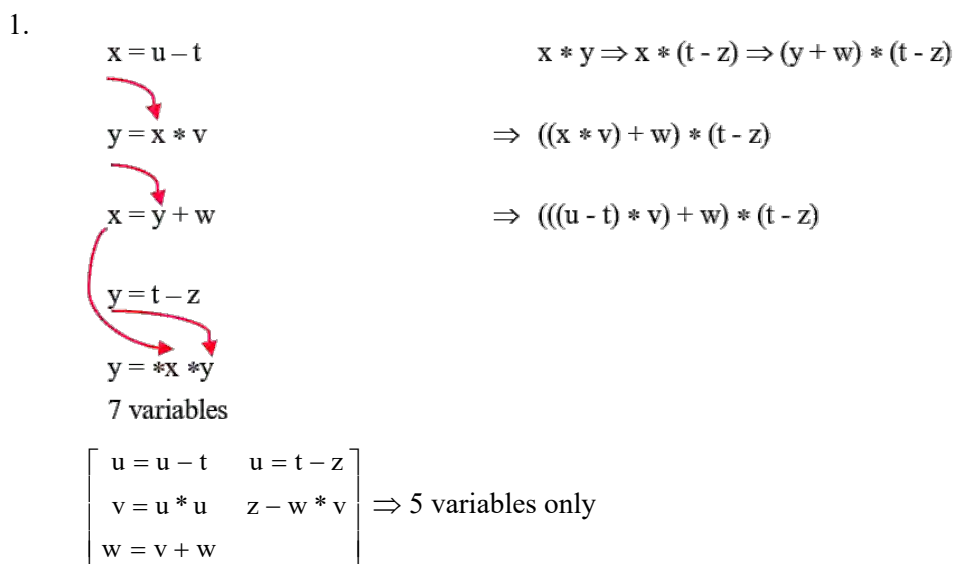


Representation of 3AC



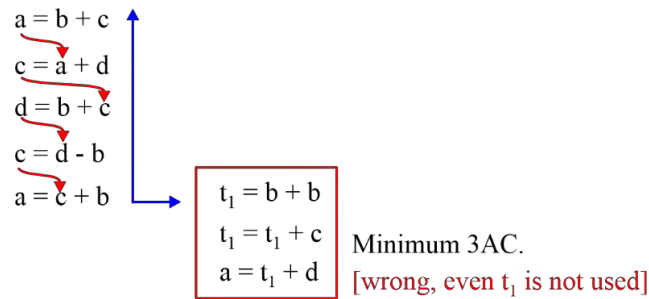
- 3AC done using operator precedence.

Find minimum number of variables required in equivalent 3AC:



⇒ Evaluating the expression:

$$\begin{aligned} a &\Rightarrow c + b \Rightarrow d - b + b \Rightarrow b + c - b + b \\ &\Rightarrow b + a + d - b + b \\ &\Rightarrow b + b + c + d - b + b \\ &\Rightarrow b + b + c + d \end{aligned}$$



∴ Minimum:

$$\left\{ \begin{array}{l} b = b + b \\ c = b + c \\ a = c + d \end{array} \right\} \Rightarrow \text{only 3 variables [most optimal]}$$

5.1.1 Static Single Assignment Code: (SSA Code)

Every variable (address) in the code has single assignment [single meaning] + 3AC.

1. $x = u - t$
 $y = x * u$
 $x = y + w$
 $y = t - z$
 $y = x * y$

Find SSA?

⇒ $[u, t, v, w, z]$ are already assigned so we can't use them.

Equivalent SSA Code:

$x = u - t$ $y = x * u$

$p = y + w$

$q = t - x$

$r = p * q$

in use: $x, y, p, q, r \Rightarrow$ additional

∴ Total Variable ⇒ 10.

2. $p = a - b$

$q = p * c$

$p = u * v$

$q = p + q$

⇒ $[a, b, c, u, v]$ are already assigned,

Equivalent SSA Code:

$p = a - b$

$q = p * c$

$p_1 = v * u$

$q_2 = p_1 + q$

in use: p, q, p_1, q_2

∴ Total Variable = 9

5.1.2 Control Flow Graphs

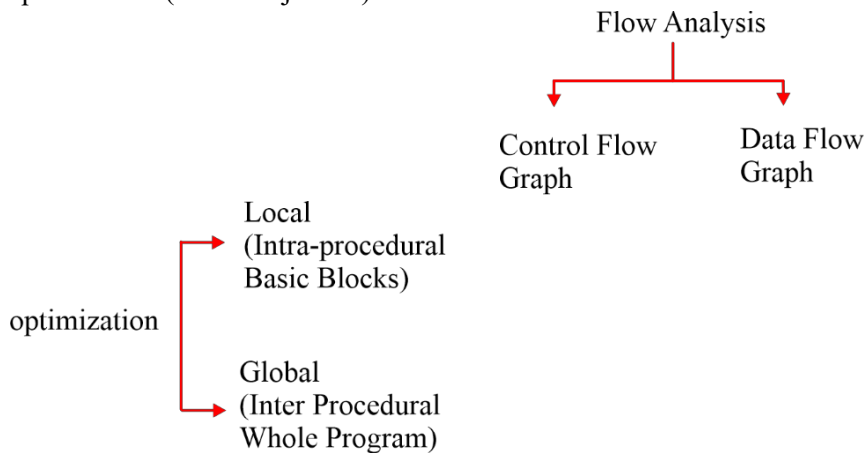
- CFG contain group of basic blocks and controls. CFG has nodes and edges to define basic blocks and controls.
- Basic Blocks: Sequence of 3-address code statements, in which control enters only from 1st statement (called as leader), and leaves from last statement.

Example:

1. $i = 1$	}	LB ₁
2. $j = 1$		LB ₂
3. $t1 = 5 * i$	}	LB ₃
4. $t2 = t1 + 5$		
5. $t3 = 4 * t2$		
6. $t4 = t3$		
7. $a[t4] = 1$	}	LB ₄
8. $j = j + 1$		
9. $\text{if } j \leq 5 \text{ goto } 3$	}	LB ₅
10. $i = i + 1$		
11. $\text{if } i < 5 \text{ goto } 2$	}	LB ₆

5.2 Code Optimization

Saves space / time. (Basic Objective)



5.2.1 Optimization Methods

- Constant folding
- Copy propagation
- Strength Reduction
- Dead code elimination
- Common sub expression elimination.
- Loop Optimization
- Peephole Optimization

1. Constant Folding

(i) $x = 2 * 3 + y \Rightarrow x = 6 + y$

Folding

ii) $x = 2 + y * 3$ can't fold the constant

2. Copy Propagation

i) Variable Propagation:

$x = y;$

$z = y + 2;$

$\Rightarrow z = x + 2;$

ii) Constant Propagation:

$x = 3$

$z = 3 + a;$

$\Rightarrow z = x + a$

3. Strength Reduction:

Replace expensive statement / instruction with cheaper one.

(i) $x = 2 * y$ **costly** $\Rightarrow x = y + y$; **Cheap**

(ii) $x = 2 * y \Rightarrow x = y < 1$; **Much Cheaper**

(iii) $x = y / 8 \Rightarrow x = y >> 3;$

4. Dead Code Elimination:

$x = 2$; **FALSE**

if ($x > 2$)

printf ("code"); **DEAD CODE CAN BE REMOVED**

else

printf ("optimization");

$x = 2;$
printf ("optimization");

- Hence, above dead code never executes during execution. We can always delete such code.

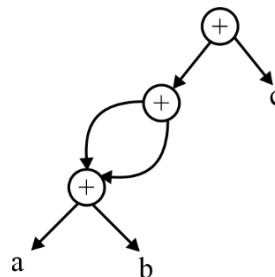
5. Common Sub Expression Elimination:

DAG is used to eliminate common sub expression.

Example: $x = (a + b) + (a + b) + c;$

$\Rightarrow t_1 = a + b$

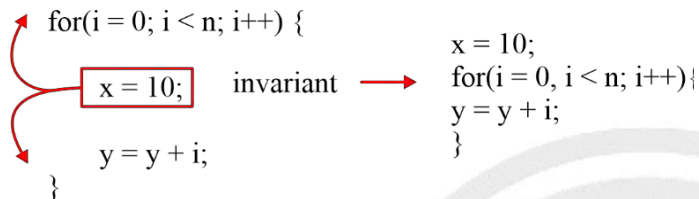
$x = t_1 + t_1 + c$



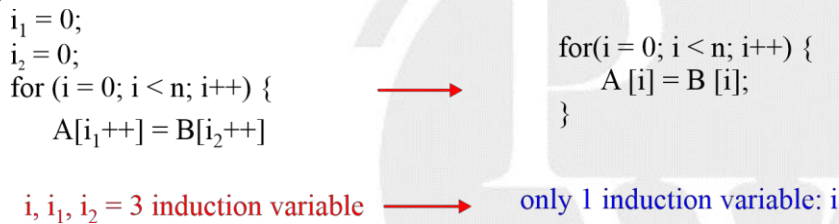
6. Loop Optimization:

(i) Code Motion – Frequency Reduction:

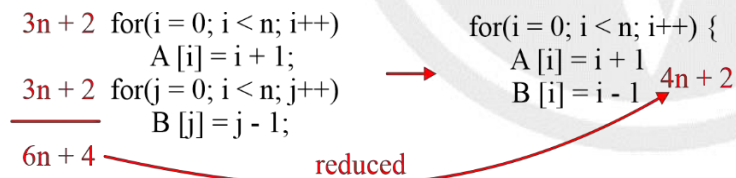
Move the loop invariant code outside of loop.



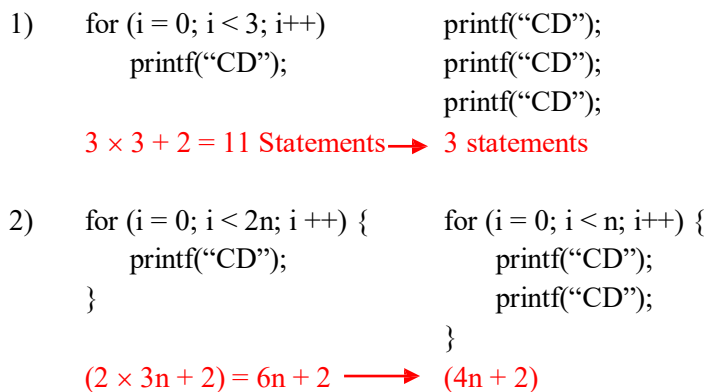
(ii) Induction Variable elimination:



(iii) Loop Merging / Combining: (Loop Jamming)



(iv) Loop Unrolling:



7. Peephole Optimization:

Examines a short sequence of target instructions in a window (*peephole*) and replaces the instructions by a faster and/or shorter sequence when possible.

- Applied to intermediate code or target code
- Following Optimizations can be used:



- Redundant instruction elimination
- Flow-of-control optimizations
- Algebraic simplifications
- Use of machine idioms



For more questions, kindly visit the library section: Link for web: <https://smart.link/sdfez8ejd80if>



PW Mobile APP: <https://smart.link/7wwosivoicgd4>